



Live Earnings Transcript — Technical Documentation

Coresight Research Portal — Technical Documentation

Document version: 1.0

Generated: June 29, 2026

Classification: Internal — Engineering Reference

CONFIDENTIAL — INTERNAL USE ONLY — DO NOT DISTRIBUTE

This document contains proprietary information belonging to Coresight Research. Unauthorized disclosure, copying, or distribution is strictly prohibited.

Table of Contents

Live Earnings Transcript — Technical Documentation	1
1. Purpose and scope	3
2. Architecture overview	3
3. Polling and refresh behavior	4
4. Database: coreiq_earnings_call_live_transcripts	5
5. Session state	7
6. Call setup (left panel)	8
7. Audio ingestion	10
8. Transcription backends	10
9. Transcript stream (right panel)	11
10. Export downloads	12
11. Dependencies	12
12. CSS	13
13. Authentication and access control	13
14. Error handling	13
15. Operational constraints (documented in UI)	14
16. Main entry flow	14
17. Comparison with Earnings Calls page	15
18. Environment variables	16
19. Related files	16
20. Testing checklist	16

Application: Market Data Portal (MDP)

Page module: `app/pages/live_earnings_transcript.py`

URL route: `/live_earnings_transcript`

Registered in: `app/main.py` as `( "pages/live_earnings_transcript.py", "Live Transcript", "live_earnings_transcript", False )`

1. Purpose and scope

The Live Earnings Transcript page is a **chunked transcription workspace** for earnings calls. It:

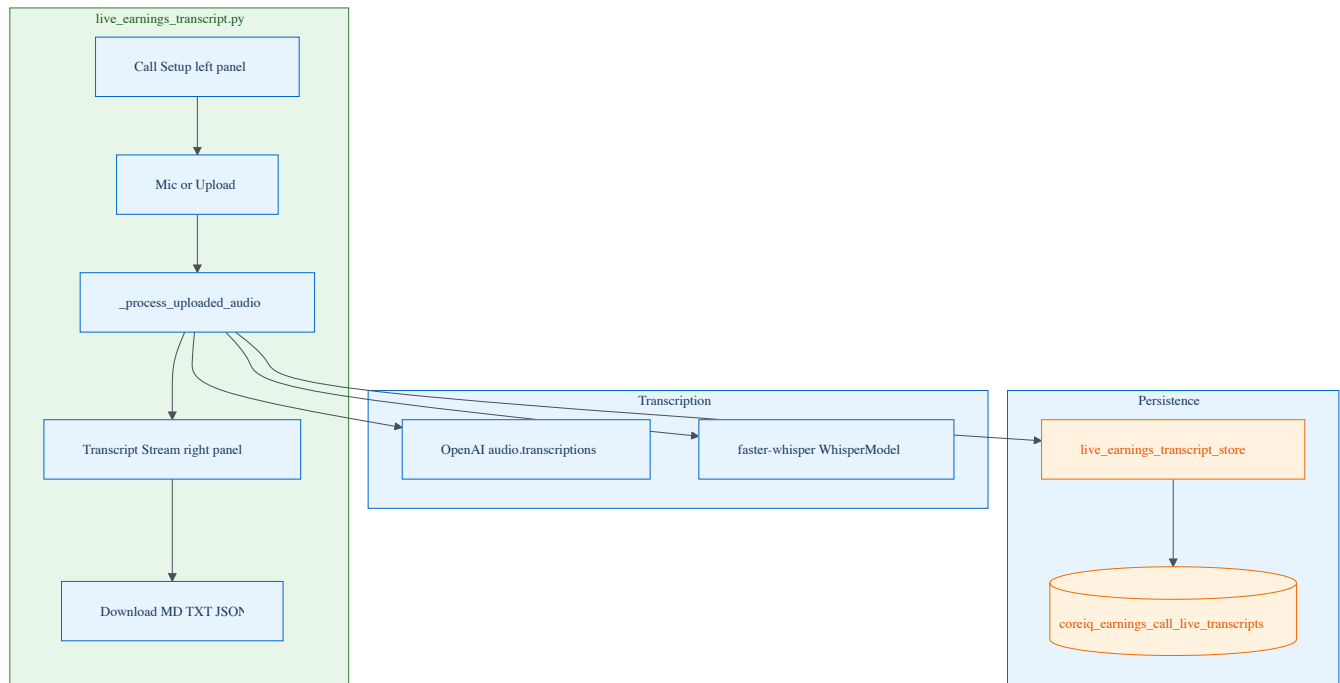
- Accepts audio via **browser microphone** (`st.audio_input`) or **file upload**
- Transcribes each completed clip with **OpenAI API** or **local faster-whisper**
- Appends segments to an in-session transcript stream
- **Persists** structured JSON to MySQL (`coreiq_earnings_call_live_transcripts`)
- Exports **MD / TXT / JSON** downloads

Important: This is **not** a live captioning or polling UI. Text appears only **after** each audio blob finishes uploading and transcription completes. There is **no auto-refresh**, **no WebSocket**, and **no periodic** `st.rerun` **polling loop** for incoming transcript text.

2. Architecture overview

Page architecture: UI orchestration, data repositories, and external services for this feature.

Page architecture: UI orchestration, data repositories, and external services for this feature.



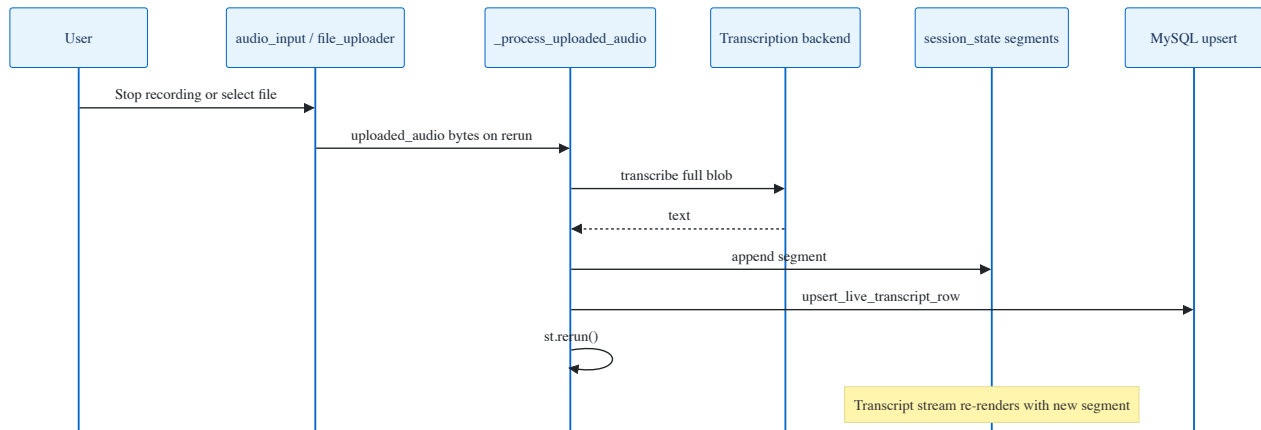
3. Polling and refresh behavior

3.1 What the page does NOT do

Expected "live" pattern	Actual behavior
Poll DB for new transcript rows	No — only writes, no reads from DB on page
<code>st.fragment(run_every=...)</code> auto-refresh	No
Stream partial text while recording	No — mic must stop/upload first
Server-side HLS/ffmpeg chunking	Planned (documented in UI expander), not implemented

3.2 What triggers UI updates

3.2 What triggers UI updates



Explicit reruns:

1. `st.rerun()` after successful transcription
2. Start / End / Clear buttons
3. "Reload run" button (mic upload pickup)
4. Environment check / clear log buttons

Timer display: `live_call_started_at` updated on Start; elapsed shown in metrics but **does not** trigger periodic refresh.

4. Database: `coreiq_earnings_call_live_transcripts`

4.1 Table usage

Written exclusively via `app/data/live_earnings_transcript_store.py` →
`upsert_live_transcript_row()`.

Column	Value source
<code>source</code>	<code>'live_streamlit' (LIVE_TRANSCRIPT_SOURCE)</code>
<code>ticker</code>	Normalized symbol (<code>normalize_ticker</code>)
<code>quarter</code>	Compact period e.g. <code>2024Q1</code> (<code>export_period_key</code>)
<code>year</code>	Fiscal year integer
<code>q</code>	1–4 from <code>Q1 – Q4</code> label
<code>transcript_text</code>	Plain text: segments joined <code>\n\n</code>
<code>has_transcript</code>	<code>1</code> if non-empty plain text
<code>title</code>	<code>{SYMBOL} {period} live transcript (max 512)</code>
<code>event_datetime_utc</code>	<code>NULL</code> (not set by Streamlit page)
<code>fetched_at_utc</code>	UTC naive timestamp
<code>raw_json</code>	Structured payload JSON
<code>raw_json_shal</code>	SHA1 of <code>raw_json</code>
<code>last_error</code>	Cleared on successful upsert

Upsert key: `(source, ticker, quarter)` — `ON DUPLICATE KEY UPDATE` refreshes all content fields.

4.2 `raw_json` schema

Built by `build_earnings_transcript_payload()` :

```
{
  "symbol": "AAPL",
  "quarter": "2024Q1",
  "transcript": [
    {
      "speaker": "",
      "title": "",
      "content": "segment text",
      "sentiment": "0"
    }
  ]
}
```

Live page segments use keys: `id`, `recorded_at`, `company`, `quarter`, `fiscal_year`, `backend`, `model`, `text` in session — mapped to `content` in DB payload (speaker/title empty unless extended later).

5. Session state

Initialized in `_init_state()`:

Key	Type	Purpose
<code>live_call_active</code>	<code>bool</code>	Timer pill "on" (not live captions)
<code>live_call_started_at</code>	<code>float</code> <code>None</code>	<code>time.time()</code> epoch for elapsed metric
<code>live_transcript_segments</code>	<code>List[Dict]</code>	All transcribed segments
<code>live_processed_audio_hashes</code>	<code>set</code>	SHA256 digest prefix dedupe
<code>live_audio_widget_version</code>	<code>int</code>	Bumps to reset audio widgets after success
<code>live_activity_log</code>	<code>List[str]</code>	Pipeline log (max 100 lines)

5.1 Segment dict shape

```
{
  "id": digest,          # sha256 hex[:20]
  "recorded_at": "YYYY-MM-DD HH:MM:SS",
  "company": "AAPL",
  "quarter": "Q1",
  "fiscal_year": 2024,
  "backend": "OpenAI API" | "Local faster-whisper",
  "model": "gpt-4o-transcribe" | "small.en" | ...,
  "text": "transcribed content",
}
```

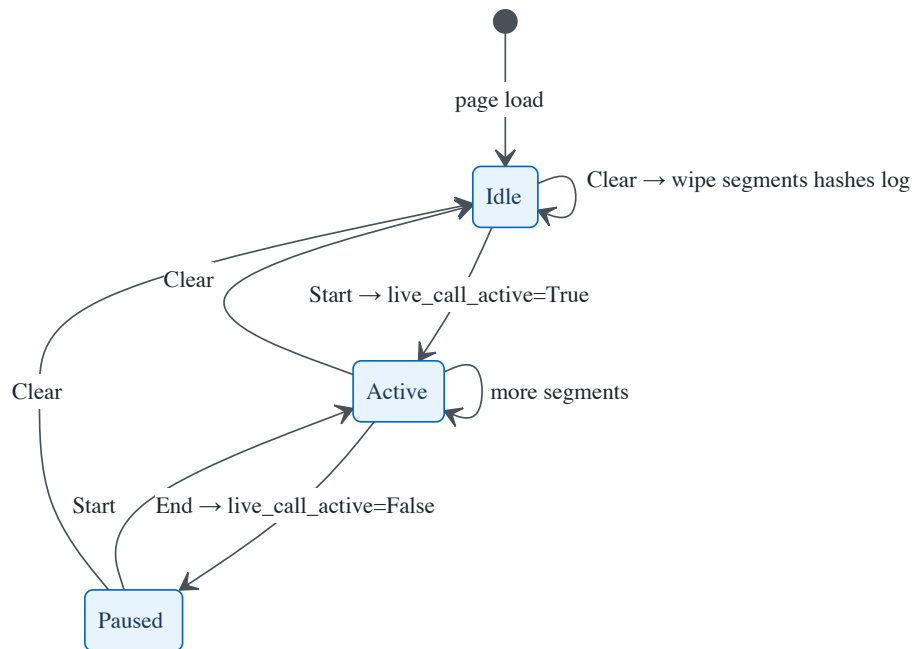
6. Call setup (left panel)

6.1 Metadata inputs

Field	Widget	Default
Company / ticker	<code>st.text_input</code>	URL <code>ticker</code> or <code>active_ticker</code>
Quarter	<code>st.selectbox</code> Q1-Q4	Q1
Fiscal year	<code>st.number_input</code>	Current calendar year
Transcription backend	selectbox	<code>Local faster-whisper</code>
OpenAI model	conditional	<code>gpt-4o-transcribe</code>
faster-whisper model	conditional	<code>small.en</code>
Device	conditional	<code>auto</code> → cuda if torch else cpu
Compute type	conditional	<code>int8</code>
Language	text	<code>en</code>
Call context	textarea	Speaker/product hints for prompt

6.2 Control buttons

6.2 Control buttons



Button	Action
Start	<code>_start_call()</code> — sets active + started_at if missing; <code>st.rerun()</code>
End	<code>_end_call()</code> — active=False only
Clear	<code>_clear_call()</code> — full reset + increment widget version

7. Audio ingestion

7.1 Modes

Mode	Constant	Widget
Microphone	<code>INGESTION_MIC</code>	<code>st.audio_input</code> key <code>live_audio_input_{version}</code>
Upload	<code>INGESTION_UPLOAD</code>	<code>st.file_uploader</code> wav/mp3/m4a/webm/ogg/flac

7.2 Processing gate

When `uploaded_audio` is not `None` :

1. Read bytes via `getvalue()`
2. Dedupe by `_audio_digest` (SHA256[:20])
3. Build domain prompt via `_build_transcription_prompt`
4. Run transcription inside `st.status` OR `st.spinner`
5. On success: append segment, upsert DB, bump widget version, `st.rerun()`

Note: Transcription runs even if user clicked **End** before upload completed (comment in code — avoids dropping in-flight clips).

7.3 Format detection

`_guess_audio_suffix()` — extension from filename or magic bytes (RIFF/WAVE, OggS, ftyp → webm).

7.4 Large file warning

Clips > 40 MB log warning in pipeline (proxy/runtime limits).

8. Transcription backends

8.1 OpenAI API

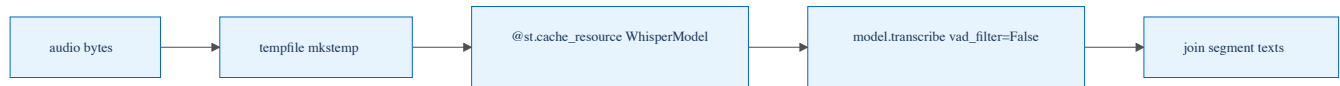
8.1 OpenAI API



- Requires `OPENAI_API_KEY`
- Models: `gpt-4o-transcribe`, `gpt-4o-mini-transcribe`, `whisper-1`
- Optional `language`, `prompt` kwargs

8.2 Local faster-whisper

8.2 Local faster-whisper



- `@st.cache_resource` on `_load_faster_whisper_model` — model loaded once per size/device/compute_type
- `vad_filter=False` — helps short browser chunks classified as non-speech
- Temp file required (BytesIO breaks webm format detection)

8.3 Shared prompt

`_build_transcription_prompt` instructs model to preserve financial terms, ticker symbols, KPIs, and optional company/quarter/context from UI.

9. Transcript stream (right panel)

9.1 Metrics bar

Metric	Source
Segments	<code>len(segments)</code>
Words	sum of <code>_word_count</code> per segment
Elapsed	<code>_elapsed_label(live_call_started_at)</code> — static until rerun

9.2 Segment rendering

`_render_transcript()` — HTML `.segment` blocks with meta:

Segment NN | recorded_at | backend | model

Empty state explains no live rolling transcript during recording.

9.3 Pipeline log

- Expander auto-expands if recent lines contain FAIL/EXCEPTION/ABORT/ERROR
- **Run environment check** — imports faster_whisper, av, OPENAI_API_KEY, `db_manager.health_check()`
- **Clear log** — resets `live_activity_log`

10. Export downloads

When `segments` non-empty:

Button	Content	MIME
Download MD	<code>_transcript_markdown</code>	text/markdown
Download TXT	<code>_transcript_text</code> numbered blocks	text/plain
Download JSON	<code>build_earnings_transcript_payload</code>	application/json

Filename base: `{TICKER}_{YYYYQn}_live_transcript`

11. Dependencies

Module	Role
<code>components.styles</code>	Layout, sidebar hide
<code>components.navigation</code>	Header, footer
<code>core.auth_manager</code>	<code>require_auth</code> optional at page level
<code>data.live_earnings_transcript_store</code>	DB upsert, payload builders
<code>core.database.db_manager</code>	Health check in env verify
<code>openai</code>	Optional OpenAI backend
<code>faster_whisper</code>	Optional local backend
<code>torch</code>	CUDA detection for device=auto

12. CSS

`_render_css()` — `.live-wrap`, metrics grid, `.segment` left border brand red `#d62e2f`, responsive breakpoints at 900px.

Status pill:

- Inactive: "Timer off"
- Active: "Timer on (not live captions)" with green styling

13. Authentication and access control

Check	Status
<code>require_auth(page="live_earnings_transcript")</code>	Optional at page level
<code>AccessControlManager.check_access</code>	Not used
DB writes	Any session user can upsert rows to <code>coreiq_earnings_call_live_transcripts</code> if the page is reachable

Identity is established via `main.py` cookie hydrate. Enable `require_auth()` at page level if live-transcript ingestion should be restricted to authenticated analysts.

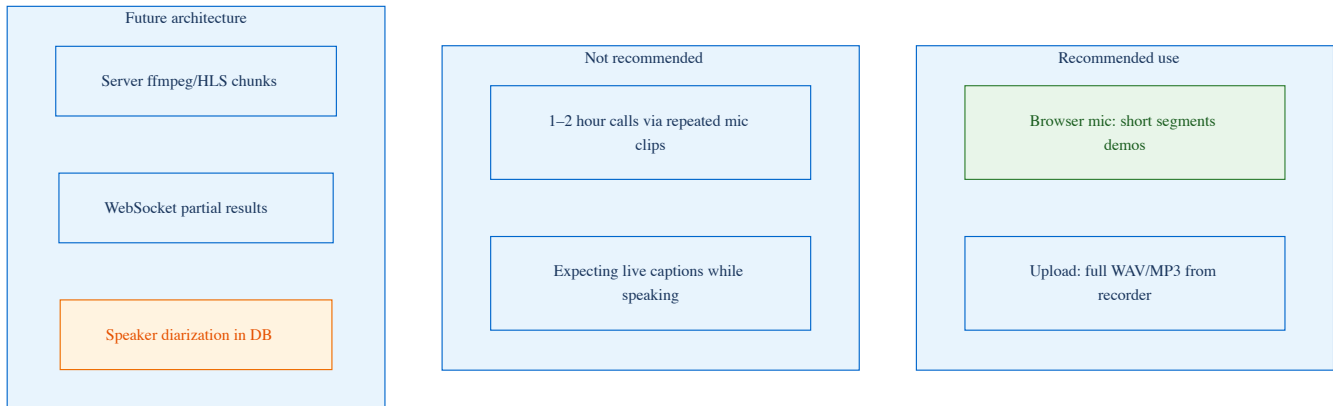
14. Error handling

Failure	UX
Zero audio bytes	Warning + pipeline ABORT log
Duplicate digest	Info skip message
Empty model text	Warning with troubleshooting tips
DB upsert fail	Warning; segment kept in session
Exception in transcribe	<code>st.error</code> + traceback expander + <code>log_structured_error</code>

`main()` top-level catch: generic refresh message.

15. Operational constraints (documented in UI)

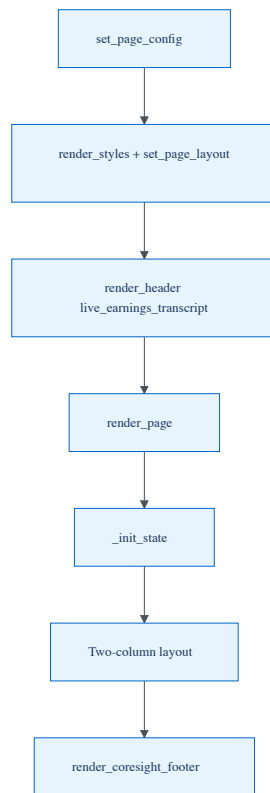
15. Operational constraints (documented in UI)



16. Main entry flow

`main()` entry point: bootstrap UI, read session/query state, fetch data, and render widgets.

main() entry point: bootstrap UI, read session/query state, fetch data, and render widgets.



Header ticker: query `ticker` or `active_ticker` or fallback `"M"`.

17. Comparison with Earnings Calls page

Aspect	<code>earnings_calls.py</code>	<code>live_earnings_transcript.py</code>
Data source	Pre-ingested AV / Azure PDF	User audio → transcription
DB table	<code>coreiq_av_earnings_call_transcripts</code>	<code>coreiq_earnings_call_live_transcripts</code>
Display	Full historical transcript	Growing segment list
Search	Keyword / FULLTEXT	None
Polling	None	None

18. Environment variables

Variable	Required for
<code>OPENAI_API_KEY</code>	OpenAI backend
<code>STG_DB_*</code> / <code>PROD_DB_*</code>	MySQL upsert (via app config)

19. Related files

File	Role
<code>app/data/live_earnings_transcript_store.py</code>	INSERT upsert SQL, JSON schema
<code>requirements.txt</code>	faster-whisper, openai optional deps
<code>app/core/database.py</code>	<code>db_manager.execute_insert</code>

20. Testing checklist

1. Upload small WAV → segment appears, pipeline log shows byte count and timing
2. Repeat same file → duplicate skip message
3. **Clear** → segments and log empty
4. Run environment check → faster_whisper, av, MySQL lines
5. Verify row in `coreiq_earnings_call_live_transcripts` for ticker/quarter key
6. Download JSON matches `raw_json` column structure